Class Test 08

Instructions:

1. **Write a Summary (Softcopy, PDF Format) on ExceptionHandlingTutorial_Spring2024-2025 and DatabaseConnectionTutorial.**
2. **Submit pdf in link provided in your VUES Student Account**
3. **The name of PDF must be your ID**

Start from here: ⟶

Summary on ExceptionHandlingTutorial_Spring2024-2025:

Exception handling in PL/SQL is a vital mechanism for managing errors and ensuring robust database applications. An exception is an identifier in PL/SQL raised during program execution, triggered either automatically by Oracle errors or explicitly by the programmer. PL/SQL offers two primary methods to handle exceptions: trapping them with a handler within the block or propagating them to the calling environment. The exception-handling section, marked by the EXCEPTION keyword, supports multiple WHEN clauses to address specific exceptions, with the WHEN OTHERS clause serving as a catch-all, always placed last. Exceptions fall into three categories: predefined Oracle Server exceptions, such as NO_DATA_FOUND and TOO_MANY_ROWS, which are automatically raised for common errors; non-predefined Oracle Server exceptions, which require the programmer to declare an exception and link it to an error code using PRAGMA EXCEPTION_INIT, as seen in handling integrity constraint violations like error -2292; and user-defined exceptions, which are explicitly declared, raised, and handled by the programmer for custom conditions. Trapping exceptions follow a structured process: predefined exceptions are referenced by name in the handler, non-predefined ones involve declaration and association with error codes, and user-defined exceptions require explicit raising in the executable section. PL/SQL provides utility functions like SQLCODE and SQLERRM to capture error codes and messages, respectively, enhancing error logging and debugging, especially within a WHEN OTHERS handler. Exceptions unhandled in the current block propagate to enclosing blocks or calling environments—such as SQL*Plus, Procedure Builder, or Oracle Developer Forms—each handling errors differently based on their context. The RAISE_APPLICATION_ERROR procedure further empowers developers by allowing custom error messages with user-defined error numbers, invocable from both executable and exception sections of stored subprograms,

aligning with Oracle's error-reporting standards. This comprehensive approach to exception handling ensures PL/SQL applications are resilient and maintainable, adept at managing both expected and unforeseen errors. Overall, the tutorial provides a clear and structured approach to handling exceptions in PL/SQL programming.

Summary on DatabaseConnectionTutorial:

The Database Connection Tutorial from the Advanced Database Management System course provides a brief guide on how to establish a connection to an Oracle Database using two popular programming languages: C# and PHP. The tutorial outlines the learning objectives clearly, aiming to equip us with practical skills for database connectivity in real-world applications.

The section on connecting from C# is a part of a voluntary community contribution linked to a previous course project titled Babysitter Service Management System. This part likely includes steps such as importing necessary libraries (like Oracle.DataAccess.Client or Oracle.ManagedDataAccess.Client), specifying the connection string, and using OracleConnection objects to open, manage, and close connections to the Oracle database.

The section on connecting from PHP is also as a community service contribution under a project named Planetary Management System. This would typically involve using functions such as oci_connect() or PDO_OCI for establishing connections, along with examples of executing queries and handling database interactions in PHP.

Overall, this tutorial, though concise, serves as a foundational reference for students to understand and implement Oracle database connectivity using C# and PHP, reinforced by examples drawn from previous student projects. It highlights the importance of practical integration between database systems and programming languages in application development.